Project Description

This project aims to build a **real-time traffic analytics pipeline** that collects live traffic data from the **TomTom Traffic Flow API**, processes it through a streaming architecture, and visualizes key metrics on a dashboard. The system demonstrates core data engineering concepts including API integration, message brokering with Kafka, time-series storage with TimescaleDB, and batch orchestration with Apache Airflow.

The pipeline ingests traffic data (current speed, free flow speed, road closures, confidence levels) at regular intervals, stores raw data immutably, performs real-time aggregations, and prepares data for visualization and alerting. The final deliverables include a live Power BI dashboard, automated notifications for traffic anomalies, and a fully documented codebase.

Group Members & Roles:

Member Name	Primary Responsibilities	Secondary Contributions
Shahd Ashraf	• Overall project coordination and sprint management • **Infrastructure setup** – Docker Compose (Kafka, PostgreSQL, Airflow), GitHub repository configuration • **Producer development** – TomTom API integration, error handling, logging, rate limit management • **Metrics calculations** – congestion ratio, statistical summaries • Code quality enforcement and documentation oversight	• Support database schema finalization • Help with testing and debugging across components • Contribute to EDA notebooks
Abdelrahman Shoukry	• **Airflow DAG development** – batch job orchestration, scheduling, task dependencies • **Windowed aggregation implementation** – time-based grouping, average speed calculations • **Kafka consumer development** – core streaming logic, message consumption • Consumer logging and error handling	• Help with producer rate limiting implementation • Database query optimization and indexing
Omnya Mamdouh	• **TomTom API integration (producer Phase 1)** – JSON parsing, data transformation • **Producer Phase 2 enhancements** – retry logic, graceful error handling, performance monitoring • **Environment variable management** – .env configuration, secrets handling • **Data validation scripts** – schema validation, quality checks	• Assist with consumer deserialization logic • Help with test script development • Assist with Airflow DAG testing
Ebtihal Ahmed	• **Database schema design** – raw_events and aggregated tables	• Help with producer testing and data insertion verification

	(traffic_aggregates, predictions) • **TimescaleDB hypertable configuration** – partitioning, indexing • **SQL query development** – analytical queries for dashboard and validation • **Power BI dashboard design** – data connections, visualizations, layout • Query performance tuning	• Assist with consumer windowed aggregation logic • Assist with data validation scripts
Aya Abdallah	• **Alert system development** – notification mechanism (email/Slack/Teams) • **Data validation framework** – missing data checks, anomaly detection thresholds • **Test script development** – Kafka connectivity tests, PostgreSQL verification • **EDA notebooks** – exploratory data analysis, data profiling • Quality assurance and end-to-end testing	• Assist with dashboard layout and design • Final documentation and report compilation • Help with Airflow DAG testing

Teamwork Philosophy:

- **All members** will participate in code reviews for every pull request.
- **Pair programming** sessions will be used for complex tasks (e.g., consumer development, Airflow DAGs, dashboard design).
- **Daily stand-ups** (15 minutes) to share progress and blockers.
- **Knowledge sharing sessions** after each phase to ensure everyone understands the entire pipeline.
- **Rotating task assignments** to prevent silos and promote cross-training.

Team Leader : <u>Shahd Ashraf</u> will serve as the team leader

Objectives

1. **Build a reliable data ingestion pipeline** that fetches traffic data from the TomTom API at configurable intervals and handles rate limits gracefully.
2. **Implement a streaming architecture** using Apache Kafka to decouple data production from consumption.
3. **Store data efficiently** in TimescaleDB (PostgreSQL with time-series extension) with proper schema design for analytical queries.
4. **Process data in real-time** using a Python consumer that computes aggregated metrics (average speed, congestion ratio) over time windows.
5. **Create a live dashboard** in Power BI connected directly to PostgreSQL for real-time traffic monitoring.
6. **Implement an alert system** that sends notifications (e.g., via email, messaging platform, or dashboard monitor) when predefined thresholds are exceeded, such as road closures or sudden speed drops.
7. **Automate batch jobs** using Apache Airflow to refresh dashboard tables and run periodic aggregations.
8. **Deliver comprehensive documentation** including setup guides, architecture diagrams, and a final project report.

Tools & Technologies

Category	Tools	Purpose
Programming Language	Python 3.10+	Core development for producers, consumers, and scripts
Data Source	TomTom Traffic Flow API	Real-time traffic data provider
Message Broker	Apache Kafka	Stream data between producer and consumer
Database	PostgreSQL with TimescaleDB	Time-series optimized storage
Containerization	Docker & Docker Compose	Consistent development environment
Orchestration	Apache Airflow	Batch job scheduling (daily aggregations, table refresh)
Visualization	Microsoft Power BI	Live dashboard
Alerting	Notification service (e.g., SMTP, Slack webhooks, or other)	Send alerts for critical events
Version Control	Git & GitHub	Code collaboration and versioning
Other Libraries	requests, kafka-python, psycopg2, python-dotenv	API calls, Kafka integration, database connectivity
Development Tools	VS Code, Jupyter Notebooks	Coding and experimentation
Project Management	JiraSoftware	Task tracking and sprint planning

Milestones & Deadlines

Milestone	Description	Deadline
Phase 0: Environment Setup	Git repository created, Docker containers (Kafka, PostgreSQL) running, team environment verified	Week 1
Phase 1: Data Ingestion	TomTom API producer implemented, data flowing to PostgreSQL and Kafka	Week 2-3
Phase 2: Stream Processing	Kafka consumer built, real-time aggregations working, metrics stored in DB	Week 5-7
Phase 3: Dashboard & Alerts	Power BI dashboard connected, alert system sending notifications	Week 8-9
Phase 4: Batch Orchestration	Airflow DAGs for daily aggregation and table refresh implemented	Week 10-11
Phase 5: Integration & Polish	Full system tested, documentation complete, final report delivered	Week 12
Final Submission	Project proposal, code repository, demo video, and final report submitted	Week 13

Key Performance Indicators (KPIs)

1. Data Preprocessing (Python script, cleaned CSV / API data)

KPI	Target	Measurement Method
API data successfully ingested	100%	Monitor producer logs – all scheduled API calls should complete without errors
Data completeness	≥ 98%	Percentage of API responses successfully parsed and stored in raw_events table
Duplicate record prevention	0% duplicates	Primary key constraint on raw_events.id ensures no duplicates; verify with SQL query
Producer uptime	≥ 99%	Log monitoring – producer should run continuously without crashing
Rate limit compliance	100%	Ensure daily API calls ≤ 2,500 (TomTom free tier); log warnings if approaching limit

2. SQL Integration (Schema, queries)

KPI	Target	Measurement Method
Schema correctness	100%	All required tables (raw_events, traffic_aggregates, anomalies, predictions) exist with correct columns
Query accuracy	≥ 95%	Test queries against known data return expected results (e.g., average speed calculations match manual computation)
Query performance	< 1 second	Average execution time for common analytical queries (e.g., SELECT with time filters)
TimescaleDB hypertable	100%	raw_events and other time-series tables created as hypertables (verify with SELECT * FROM timescaledb_information.hypertables)

3. Visualization (Charts, dashboard) & Alerts

	Target	Measurement Method
Dashboard load time	< 3 seconds	Time from opening Power BI dashboard to data appearing (using DirectQuery)
Metrics visualized	≥ 90%	At least 9 of the following 10 metrics are displayed: current speed, free flow speed, congestion ratio, average speed (time-series), road closure alerts, anomaly count, confidence level, location filter, time filter, alert history
Dashboard refresh rate	≤ 1 minute	Dashboard updates automatically when new data arrives (DirectQuery mode)
Alert delivery success rate	100%	Every threshold breach triggers a notification (verified by manual testing)
Alert response time	< 1 minute	Time from event occurrence to notification (measured in logs)
User satisfaction	≥ 4/5	Feedback from team/stakeholders on dashboard usability and clarity

4. Presentation (Report, slide deck)

KPI	Target	Measurement Method
Report completeness	100%	All required sections present: project description, architecture, tools, milestones, KPIs, team roles, challenges, future work
Documentation quality	≥ 4/5	Clear, well-structured README, SETUP guide, and project documentation with diagrams
Demo video	Included	5-minute video demonstrating the working pipeline, dashboard, and alerts
Stakeholder clarity	≥ 4/5	Feedback from instructor/mentor on presentation clarity and depth
Code quality	≥ 90%	All code passes pre-commit hooks (black) and includes comments/docstrings